

RAPPORT DE PROJET

Application Web de Gestion de Stock

PHP / MySQL / PDO

Formation : BTS SIO / BUT Informatique

Année scolaire : 2025 – 2026

Date : Avril 2026

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 1
Nom, prénom : CAREME Joey		N° candidat : 02051928866
Épreuve ponctuelle ☐	Contrôle en cours de formation ☒	Date : / /
Organisation support de la réalisation professionnelle		
Lycée Melkoir Garré — BTS SIO option 2 SLAM		
Intitulé de la réalisation professionnelle : Stokio		
Période de réalisation : De Janvier 2025 à avril 2025 Lieu : Cayenne		
Modalité : ☒ Seul(e) ☐ En équipe		
Compétences travaillées		
☒ Concevoir et développer une solution applicative ☐ Assurer la maintenance corrective ou évolutive d'une solution applicative ☐ Gérer les données		
Conditions de réalisation¹ (ressources fournies, résultats attendus)		
Ressources fournies : poste de travail personnel équipé d'un serveur local XAMPP (Apache + MySQL), éditeur de code VS Code, navigateur web. Résultats attendus : application web fonctionnelle de gestion de stock avec authentification par rôle, opérations CRUD sur les produits, recherche et filtrage dynamique, et gestion des utilisateurs réservée au superadmin.		
Description des ressources documentaires, matérielles et logicielles utilisées²		
Matérielles : ordinateur personnel, serveur local Apache/MySQL via XAMPP. Logicielles : PHP 8, MySQL, PDO, VS Code, navigateur web (Chrome/Firefox).		
Modalités d'accès aux productions³ et à leur documentation⁴		
Fichiers sources PHP, CSS et SQL disponibles localement sur le poste du candidat. Accès via serveur local XAMPP : http://localhost/gestion_stock/ Identifiants de test : superadmin / supadmin — admin / admin123 — employe1 / test123.		

¹ En référence aux conditions de réalisation et ressources nécessaires du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

² Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

³ Conformément au référentiel du BTS SIO « Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des »

Épreuve E5 - Conception et développement d'applications (option SLAM)**ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)****Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs**

Stokio est une application web de gestion de stock développée en PHP avec PDO et MySQL.

Fonctionnalités :

- Authentification sécurisée : 3 rôles (superadmin, admin, employe) session PHP.
- CRUD complet sur les produits : ajout, modification, suppression, consultation.
- Recherche par nom/référence et filtrage par catégorie (menu déroulant dynamique).
- Gestion des utilisateurs réservée au superadmin (ajout, suppression, protection du compte superadmin).
- Réinitialisation du mot de passe via formulaire dédié.

Base de données : 2 tables relationnelles (produits, utilisateurs) — script SQL fourni.

Sécurité : requêtes préparées PDO, htmlspecialchars(), hashage bcrypt (password_hash), contrôle de session.

Interface : HTLM/CSS responsive, thème sombre moderne, animations CSS

interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve. ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

⁴ Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

1. Introduction & Contexte

1.1 Présentation du projet

Ce projet a été réalisé dans le cadre d'une formation en informatique BTS SIO SLAM. Il consiste en la conception et le développement d'une application web complète de gestion de stock, accessible depuis un navigateur et sécurisée par un système d'authentification.

L'objectif principal est de permettre à une structure (entreprise, établissement, association) de suivre en temps réel l'état de son inventaire : ajout, modification, suppression et consultation de produits, avec une gestion des droits d'accès par rôle utilisateur.

1.2 Objectifs

- Mettre en place une interface web intuitive pour gérer un inventaire de produits.
- Sécuriser l'accès via un système de connexion avec gestion des rôles.
- Permettre la recherche et le filtrage dynamique des produits.
- Offrir une gestion des utilisateurs réservée au superadministrateur.
- Assurer la persistance des données via une base MySQL.

1.3 Technologies utilisées

Stack technique

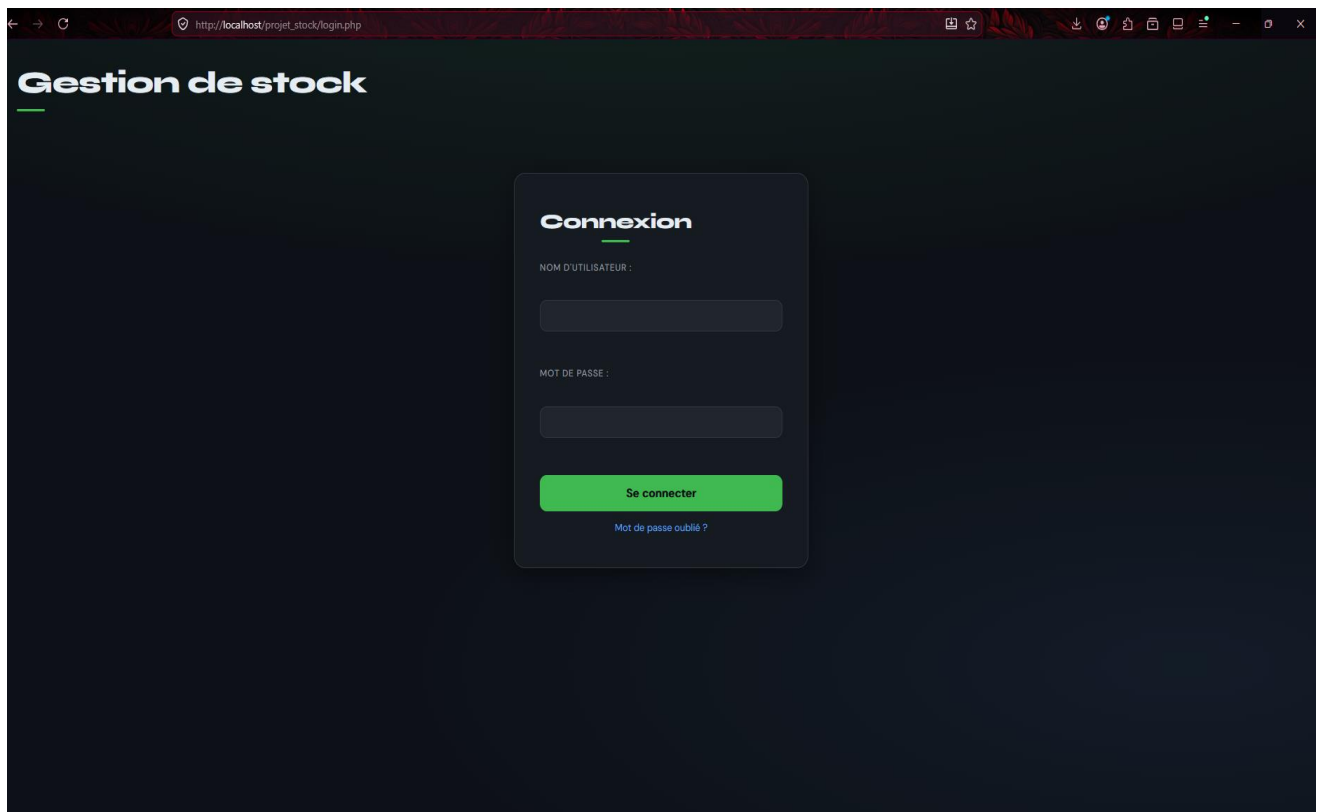
Back-end : PHP 8 | Base de données : MySQL | Front-end : HTML5, CSS3 (design responsive) | Sécurité : Sessions PHP, hachage SHA-256 / password_hash

2. Fonctionnalités Détaillées

2.1 Authentification (login.php)

L'accès à l'application est conditionné à une connexion obligatoire. Le formulaire de connexion collecte le login et le mot de passe, puis vérifie les identifiants en base de données. En cas de succès, les informations de l'utilisateur (id, login, rôle) sont stockées en session.

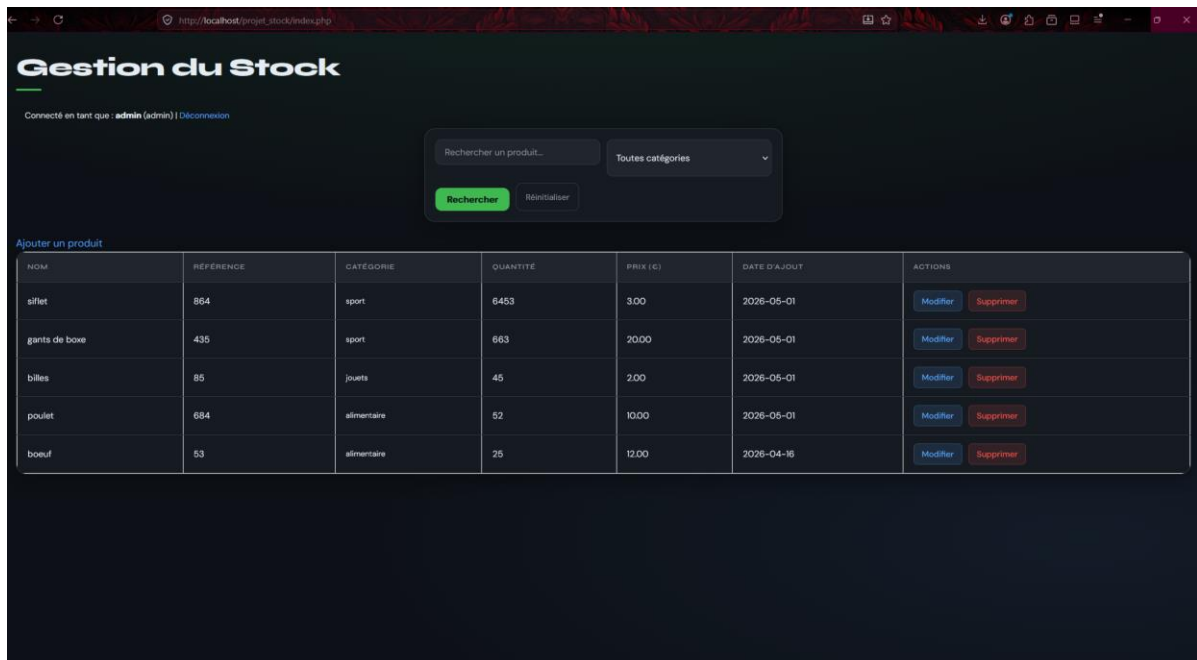
- Vérification des identifiants via SHA-256.
- Redirection automatique vers index.php après connexion.
- Message d'erreur affiché si les identifiants sont incorrects.
- Lien vers la page de réinitialisation du mot de passe.



2.2 Tableau de bord – Liste des produits (index.php)

Page principale de l'application. Elle affiche l'ensemble des produits stockés dans la base de données sous forme de tableau. L'utilisateur voit son login et son rôle en haut de page.

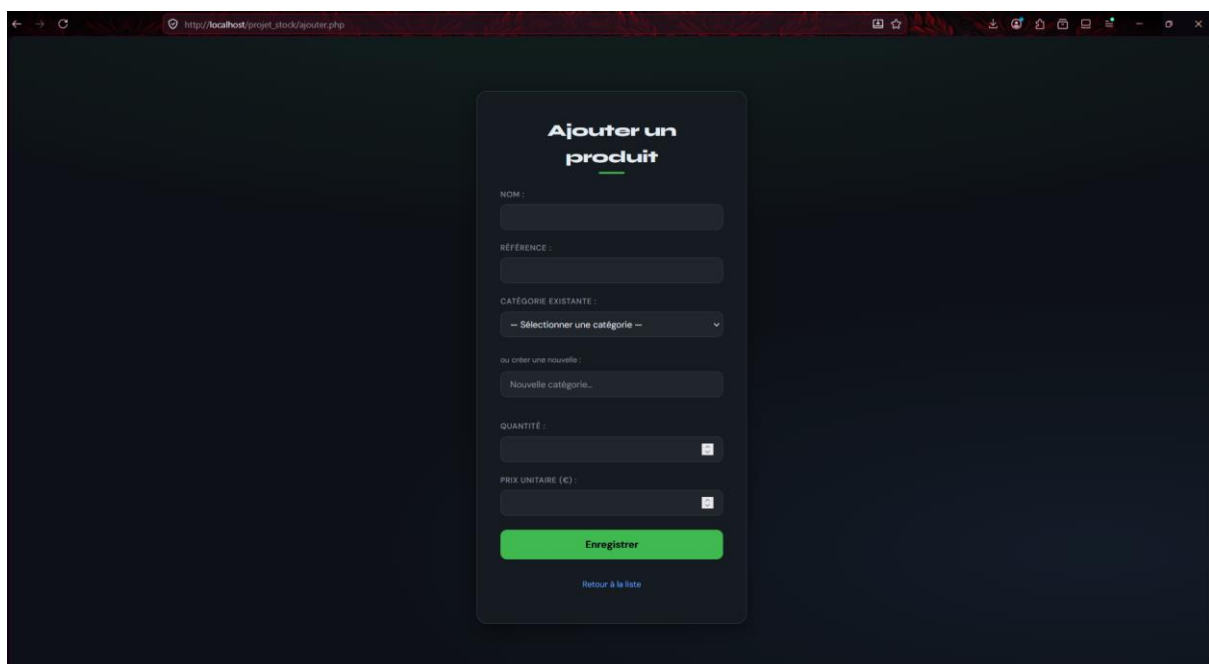
- Affichage : nom, référence, catégorie, quantité, prix unitaire, date d'ajout.
- Recherche en temps réel par nom ou référence (paramètre GET).
- Filtrage par catégorie via un menu déroulant dynamique.
- Boutons d'action : Modifier et Supprimer (conditionnés au rôle).
- Lien vers la gestion des utilisateurs réservé au superadmin.



2.3 Ajout d'un produit (ajouter.php)

Formulaire permettant d'ajouter un nouveau produit à l'inventaire. Une amélioration notable a été apportée sur la gestion des catégories.

- Champs : nom, référence, catégorie, quantité, prix unitaire.
- Catégorie : menu déroulant chargé dynamiquement depuis la base (catégories existantes réutilisables).
- Champ libre pour créer une nouvelle catégorie si elle n'existe pas encore.
- Priorité à la nouvelle catégorie saisie si les deux champs sont remplis.
- Insertion en base via une requête préparée PDO (Protected Designation of Origin / protection contre les injections SQL).



2.4 Modification d'un produit (modifier.php)

Formulaire pré-rempli avec les données actuelles du produit sélectionné. L'utilisateur peut modifier tous les champs et enregistrer les changements.

- Récupération du produit par son ID (paramètre GET).
- Vérification de l'existence du produit avant affichage.
- Mise à jour via requête UPDATE préparée.

2.5 Suppression d'un produit (supprimer.php)

Suppression d'un produit après confirmation (boîte de dialogue JavaScript). La suppression est effectuée via une requête DELETE sécurisée.

- Vérification de l'existence du produit avant suppression.
- Confirmation côté client via onclick / confirm().
- Redirection vers index.php après suppression.

```

1  <?php
2  session_start();
3  if (!isset($_SESSION['user'])) {
4      header("Location: login.php");
5      exit;
6  }
7  ?>
8
9  <?php
10 require 'config.php';
11
12 if (!isset($_GET['id'])) {
13     die("ID du produit manquant !");
14 }
15
16 $id = $_GET['id'];
17
18 // Vérifie que le produit existe
19 $stmt = $pdo->prepare("SELECT * FROM produits WHERE id = ?");
20 $stmt->execute([$id]);
21 $produit = $stmt->fetch(PDO::FETCH_ASSOC);
22
23 if (!$produit) {
24     die("Produit introuvable !");
25 }
26
27 // Supprime le produit
28 $stmt = $pdo->prepare("DELETE FROM produits WHERE id = ?");
29 $stmt->execute([$id]);
30
31 header("Location: index.php");
32 exit;
33 ?>
34

```

2.6 Gestion des utilisateurs (superadmin uniquement)

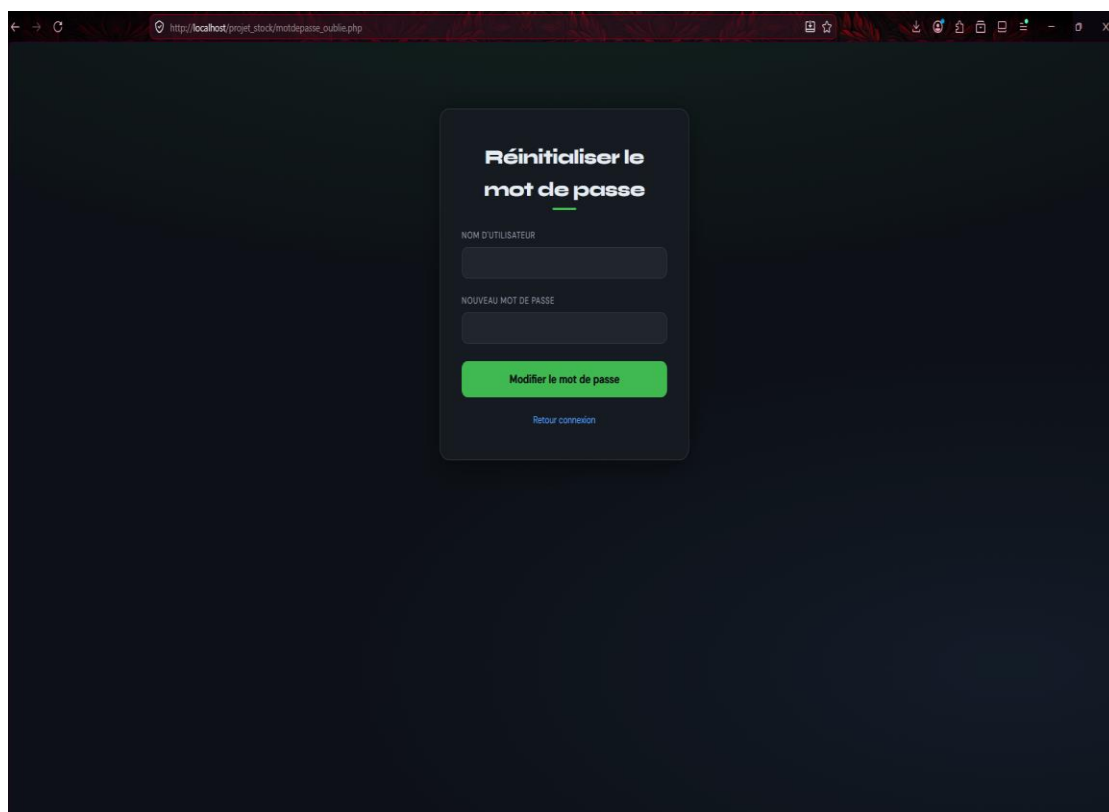
Accessible uniquement aux utilisateurs ayant le rôle superadmin, cette section permet de visualiser, ajouter et supprimer des comptes utilisateurs.

Rôle	Droits
employe	Consultation du stock uniquement
user	Consultation + recherche/filtrage
admin	CRUD complet sur les produits
superadmin	Tout + gestion des utilisateurs (ajout, suppression)

- Le compte superadmin est protégé : il ne peut pas être supprimé depuis l'interface.
- Ajout d'un utilisateur avec choix du rôle et hachage automatique du mot de passe.

2.7 Réinitialisation du mot de passe (motdepasse_oublie.php)

Page accessible sans connexion, permettant à un utilisateur de redéfinir son mot de passe en fournissant simplement son login. Le nouveau mot de passe est haché en SHA-256 avant d'être enregistré.



The screenshot shows a web browser window with the URL `http://localhost/projet_stock/motdepasse_oublie.php`. The page has a dark background. In the center, there is a light gray box containing the title "Réinitialiser le mot de passe" in bold. Below the title, there are two input fields: "NOM D'UTILISATEUR" and "NOUVEAU MOT DE PASSE". Below these fields is a green button with the text "Modifier le mot de passe". At the bottom of the box, there is a link that says "Retour connexion".

2.8 Architecture des fichiers

Fichier	Rôle
<code>config.php</code>	Connexion PDO à la base de données MySQL
<code>index.php</code>	Page principale : liste, recherche, filtrage des produits
<code>login.php</code>	Formulaire de connexion et gestion de session
<code>logout.php</code>	Destruction de la session et redirection
<code>ajouter.php</code>	Formulaire d'ajout d'un nouveau produit
<code>modifier.php</code>	Formulaire de modification d'un produit existant
<code>supprimer.php</code>	Suppression sécurisée d'un produit
<code>utilisateurs.php</code>	Gestion des comptes utilisateurs (superadmin)
<code>ajouter_utilisateur.php</code>	Création d'un nouveau compte utilisateur
<code>motdepasse_oublie.php</code>	Réinitialisation du mot de passe
<code>style.css</code>	Feuille de style CSS (thème sombre, responsive)
<code>bdd_stock.sql</code>	Script SQL de création de la base de données

3. Sécurité

3.1 Authentification et sessions

L'authentification repose sur le mécanisme de sessions PHP (`session_start()`). Chaque page sensible vérifie la présence de la variable de session `$_SESSION['user']` et redirige vers `login.php` en cas d'absence. Cela empêche tout accès direct aux pages sans connexion préalable.

Vérification systématique

Chaque fichier PHP (hors `login.php` et `motdepasse_oublie.php`) commence par : `session_start()` + vérification de `$_SESSION['user']`, garantissant qu'aucune page n'est accessible sans authentification.

```
<?php
require 'config.php';
session_start();
if (!isset($_SESSION['user'])) {
    header("Location: login.php");
    exit;
}
```

3.2 Hachage des mots de passe

Deux méthodes de hachage sont utilisées dans le projet :

- SHA-256 (`hash('sha256', ...)`) : utilisé pour la connexion et la réinitialisation du mot de passe. C'est la méthode cohérente avec les mots de passe créés dans le script SQL.
- `password_hash()` / `PASSWORD_DEFAULT` : utilisé lors de la création d'un utilisateur via l'interface. Cette fonction génère un hachage bcrypt, plus robuste face aux attaques par force brute.

Point d'attention

Il existe une incohérence dans le projet : les utilisateurs créés via `ajouter_utilisateur.php` utilisent `password_hash()`, mais la vérification dans `login.php` compare via `hash('sha256', ...)`. Ces deux méthodes étant incompatibles, les comptes créés via l'interface ne peuvent pas se connecter. Une harmonisation vers `password_verify()` / `password_hash()` est recommandée.

3.3 Protection contre les injections SQL

L'ensemble des requêtes SQL qui utilisent des données issues de l'utilisateur sont construites avec des requêtes préparées via PDO (`prepare()` + `execute()`). Cela neutralise toute tentative d'injection SQL.

- INSERT, UPDATE, DELETE, SELECT avec paramètres : tous préparés.
- Aucune concaténation directe de variables dans les requêtes SQL.

3.4 Protection XSS

Les données affichées dans les pages HTML sont systématiquement échappées via `htmlspecialchars()`, ce qui empêche l'injection de code JavaScript malveillant (attaques XSS) via les champs du formulaire ou la base de données.

3.5 Contrôle d'accès par rôle

Certaines fonctionnalités sont conditionnées au rôle de l'utilisateur connecté :

- Accès à `utilisateurs.php` et `ajouter_utilisateur.php` réservé au superadmin (vérification du rôle en session).
- Le bouton 'Gérer les utilisateurs' n'est affiché dans `index.php` que pour le superadmin.
- La suppression du compte superadmin est bloquée dans l'interface.

```
48     <?php if ($_SESSION['user']['role'] === 'superadmin'): ?>
49         <a href="utilisateurs.php">Gérer les utilisateurs</a>
50     <?php endif; ?>
```

Piste d'amélioration sécurité

La page `motdepasse_oublie.php` permet à n'importe qui de changer le mot de passe d'un compte en connaissant uniquement le login. En production, il faudrait ajouter une vérification complémentaire (email de confirmation, question secrète, token temporaire).

4. Conclusion & Perspectives

4.1 Bilan du projet

Ce projet a permis de mettre en pratique les compétences fondamentales du développement web back-end : conception de base de données relationnelle, développement PHP avec PDO, gestion des sessions et des droits d'accès, et intégration d'une interface utilisateur responsive.

L'application couvre l'ensemble du cycle CRUD (Create, Read, Update, Delete) sur les produits, avec une couche de sécurité fonctionnelle (authentification, hachage, requêtes préparées, protection XSS). La gestion des rôles apporte une dimension multi-utilisateur réaliste.

4.2 Améliorations envisageables

- **utiliser password_hash() / password_verify() de manière uniforme.** : Harmonisation du hachage
- **ajout d'un système de token par email.** : Réinitialisation du mot de passe sécurisée
- **empêcher les utilisateurs simples de modifier ou supprimer des produits (actuellement, tous les utilisateurs connectés y ont accès).** : Gestion des droits d'édition
- **limiter l'affichage à N produits par page pour les grands catalogues.** : Pagination
- **ajouter un export CSV ou PDF de l'inventaire.** : Export
- **enregistrer les modifications (qui a fait quoi et quand).** : Journal des actions
- **ne pas stocker les identifiants de base de données en clair dans config.php en production.** : HTTPS & variables d'environnement

4.3 Compétences mobilisées

- Conception et modélisation d'une base de données relationnelle (MySQL).
- Développement back-end en PHP avec PDO et gestion des sessions.
- Sécurisation d'une application web (injections SQL, XSS, contrôle d'accès).
- Création d'une interface utilisateur responsive en HTML/CSS.
- Gestion d'un projet complet de la base de données à l'interface utilisateur.

Conclusion

Ce projet constitue une base solide et fonctionnelle pour une application de gestion de stock en contexte professionnel. Avec les améliorations identifiées, notamment la sécurisation de la réinitialisation de mot de passe et l'harmonisation du hachage, l'application pourrait être déployée en environnement réel.